

Лабораторная работа №6-7. Взаимодействие с пользовательским интерфейсом и State в Jetpack Compose. Написание автоматических тестов

Цель работы

Познакомиться с понятием **состояния (state)** в Android-приложениях и научиться использовать его в интерфейсе с помощью **Jetpack Compose**. В результате лабораторной работы будет создано приложение **калькулятор чаевых (Tip Time)**, которое автоматически рассчитывает сумму чаевых. Изучить автоматизированные тесты в Android.

Теоретическая часть

State — это изменяемые данные приложения (ввод пользователя, выбор элементов, результаты вычислений).

Примеры state:

- вводимый текст;
- состояние кнопки;
- системные сообщения.

Примеры состояния в Android-приложениях:

- Сообщение об отсутствии интернет-соединения;
- Текст, вводимый в форму регистрации;
- **State** кнопки: не нажата / нажата / выполняет действие.

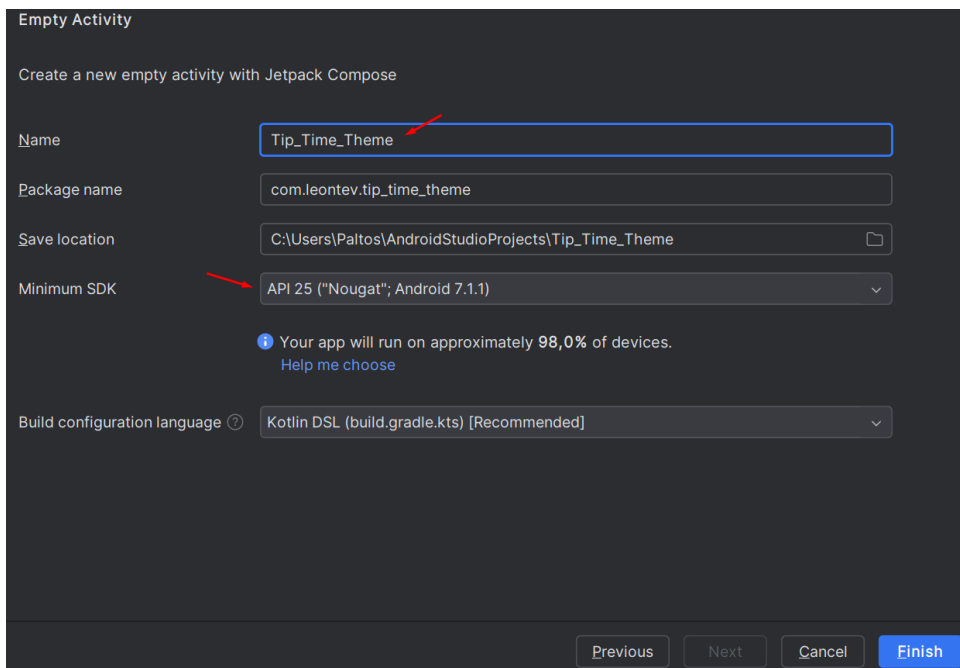
В работе используются: TextField, Text, Spacer и функция расчёта чаевых.

Практическая часть

Шаг 1. Создание проекта

1. Откройте **Android Studio**
2. Выберите **File** → **New** → **New Project**
3. В окне выбора шаблона выберите **Empty Activity** → **Next**
4. Укажите параметры:
 - **Name: Tip_Time_Theme**
 - **Minimum SDK: API 25**
5. Нажмите **Finish**

Дождитесь полной загрузки проекта.



Шаг 2. Очистка стартового шаблона

В **MainActivity.kt** удалите лишние **@Composable-функции**, оставив базовую структуру активности:

```
1 class MainActivity : AppCompatActivity() {
2     override fun onCreate(savedInstanceState: Bundle?) {
3         super.onCreate(savedInstanceState)
4         enableEdgeToEdge()
5         setContent {
6             Tip_Time_ThemeTheme {
7
8             }
9         }
10    }
11}
```

Шаг 3. Создание функции для расчёта чаевых

Создаём функцию, которая принимает сумму счёта, рассчитывает 15% и возвращает отформатированный результат.

Добавляем функцию под классом **MainActivity**:

```
private fun calculateTip(amount: Double, tipPercent: Double = 15.0): String {
    // ...
}
```

Далее внутри функции пропишем:

```
val tip = tipPercent / 100 * amount
return NumberFormat.getCurrencyInstance().format(number = tip)
```

Разбор функции: Функция принимает сумму и процент чаевых, вычисляет значение и форматирует его как валюту.

- **amount: Double** — сумма счёта;
- **tipPercent: Double = 15.0** — процент чаевых (по умолчанию 15%);
- **val tip** — здесь происходит сам расчёт;
- **NumberFormat.getCurrencyInstance()** — форматирует число как валюту.

Шаг 4. Создание интерфейса (Composable-функция)

Создаём основной интерфейс приложения на базе Column.

1. Ниже функции `calculateTip()` добавляем `@Composable` функцию:

```
@Composable
fun TipTimeLayout() {

}
```

2. В ней добавляем `Column` как контейнер:

```
@Composable
fun TipTimeLayout() {
    Column {
        // сюда будем добавлять виджеты
    }
}
```

`Column { }` будет вертикальным контейнером.

3. Добавляем модификаторы контейнера:

```
@Composable
fun TipTimeLayout() {
    Column(
        modifier = Modifier
            .statusBarsPadding()
            .padding(horizontal = 40.dp)
            .safeDrawingPadding(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        // содержимое
    }
}
```

1. Расширим **Column** модификаторами: отступы от статус-бара, внутренние отступы по горизонтали.

2. Добавим выравнивание и расположение элементов по центру.

statusBarsPadding() и **safeDrawingPadding()** защищают от наложения системы; **horizontalAlignment** и **verticalArrangement** задают центровку.

4. Добавляем заголовок (Text)

1. Внутри **Column** добавьте **Text** с заголовком "Calculate Tip".

2. Примените верхний и нижний отступ, и выровняйте текст к началу (чтобы заголовок не был по центру).

```
    ) {  
        Text(  
            text = "Calculate Tip",  
            modifier = Modifier  
                .padding(bottom = 16.dp, top = 40.dp)  
                .align(alignment = Alignment.Start)  
        )  
    }  
}
```

5. Добавляем отображение суммы чаевых (placeholder)

1. Добавьте **второй Text**, который будет показывать сумму чаевых. Пока используем статический плейсхолдер "\$0.00".

2. Примените стиль текста (например **displaySmall** или **headlineSmall** в вашей теме).

```
        Text(  
            text = "Tip Amount: $0.00",  
            style = MaterialTheme.typography.displaySmall  
        )  
    }  
}
```

Позже это поле заменим на динамическое значение, завязанное на **State**.

6. Добавляем Spacer для отступа

1. Между элементами или внизу добавьте **Spacer** чтобы визуально отделить блоки.

2. Укажите высоту 150.dp.

```

    )
    Spacer(modifier = Modifier.height(height = 150.dp))
}
}

```

Шаг 5. Добавление Preview для предварительного просмотра

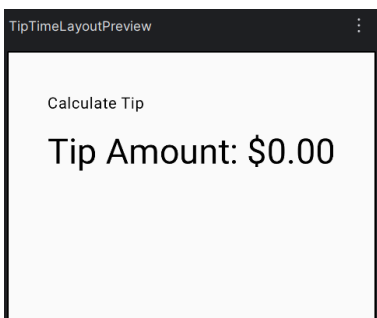
Создаём **@Preview**, чтобы просматривать экран без запуска приложения:

```

@Preview(showBackground = true)
@Composable
fun TipTimeLayoutPreview() {
    Tip_Time_ThemeTheme {
        TipTimeLayout()
    }
}

```

Теперь можно нажать **Preview** и увидеть экран приложения:



Шаг 6. Подключение интерфейса к MainActivity

Подключаем `TipTimeLayout()` как основной экран приложения. Внутри **MainActivity** должно быть:

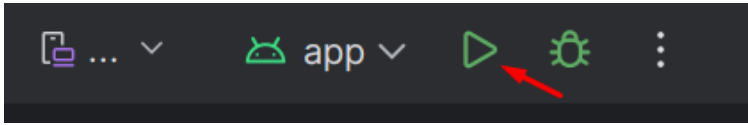
```

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            Tip_Time_ThemeTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                ) {
                    TipTimeLayout()
                }
            }
        }
    }
}

```

Шаг 7. Запуск приложения

1. Выберите эмулятор или подключённый телефон
2. Нажмите **Run**

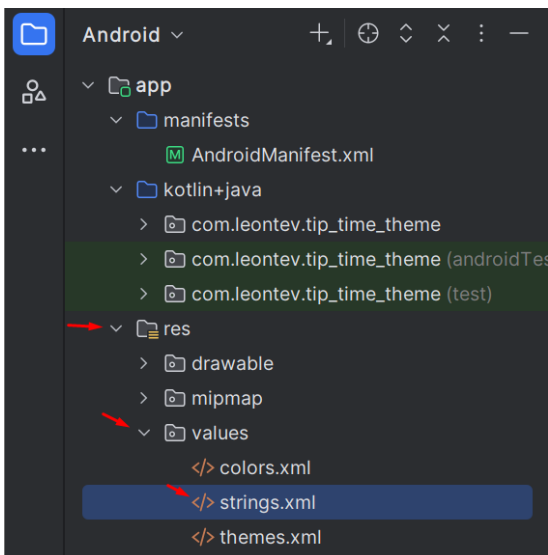


3. Убедитесь, что отображаются:
 - о заголовок **"Calculate Tip"**;
 - о текст **"Tip Amount: \$0.00"**.

Шаг 8. Подключаем строки (strings.xml)

Тексты выносятся в strings.xml и используются через `stringResource(...)`.

1. Откройте **res > values > strings.xml**



И добавьте следующие строки:

```
<resources>
    <string name="app_name">Tip_Time_Theme</string>
    <string name="calculate_tip">Calculate Tip</string>
    <string name="bill_amount">Bill Amount</string>
    <string name="tip_amount">Tip Amount: %s</string>
</resources>
```

tip_amount содержит форматный параметр `%s` — так мы сможем подставлять отформатированную сумму чаевых (например, "\$1.50").

Теперь обновим наш код добавив вызов:

2. В `TipTimeLayout()` замените «жёстко прописанные» тексты на вызовы `stringResource`:

У **первого текста** в функции **TipTimeLayout()** замените «жёстко прописанные» тексты на вызовы **stringResource**:

Было:

```
Text(  
    text = "Calculate Tip",
```

Меняем на:

```
) {  
    Text(  
        text = stringResource(id = R.string.calculate_tip),  
        modifier = Modifier
```

У **второго текста** в функции **TipTimeLayout()**

Было:

```
Text(  
    text = "Tip Amount: $0.00",
```

Меняем на:

```
Text(  
    text = stringResource(id = R.string.tip_amount, ...formatArgs = "$0.00"),
```

Не забудьте импорт:

```
import androidx.compose.ui.res.stringResource
```

Использование strings.xml упрощает локализацию и убирает жёстко заданные строки из кода.

Шаг 9. Принимаем данные от пользователя (TextField)

Цель: добавить поле ввода суммы счёта.

Стили темы уже настроены, на этом этапе важна только работа поля ввода.

9.1. Создаём компонентную функцию

В **MainActivity.kt** (вне класса **MainActivity**) добавьте заготовку функции:

```
@Composable  
fun EditNumberField(modifier: Modifier = Modifier) {  
  
}
```

Функция отвечает только за отображение **TextField**. **Modifier** вынесен в параметр для гибкой настройки.

9.2. Вставляем сам TextField

Внутри `EditNumberField()` добавьте `TextField` с минимальным набором параметров:

```
TextField(  
    value = "",  
    onValueChange = {},  
    modifier = modifier  
)
```

- **value** — текущее значение поля.
- **onValueChange** — вызывается при вводе текста пользователем.

Импорт, если он не появился:

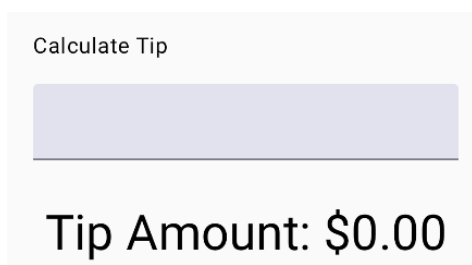
```
import androidx.compose.material3.TextField
```

9.3. Вставляем вызов в TipTimeLayout()

В `TipTimeLayout()` — сразу после заголовка — вызовите `EditNumberField(...)` с нужным модификатором. Например:

```
60 fun TipTimeLayout() {  
61     > Column(...) {  
69     >     Text(...)  
75     [ EditNumberField(modifier = Modifier  
76         .padding(bottom = 32.dp).fillMaxWidth())  
77     >     Text(...)  
81     Spacer(modifier = Modifier.height(height = 150.dp))  
82     }  
83 }
```

После добавления поля ввода интерфейс содержит заголовок, текстовое поле и статичное отображение суммы чаевых.



Шаг 10. Использование State в Compose

State — изменяемые данные приложения. В данном случае это сумма счёта, вводимая пользователем.

10.1. Создание переменной состояния

В начале функции `EditNumberField()` создайте переменную `amountInput` и присвойте ей значение `"0"`:

```
fun EditNumberField(modifie
    val amountInput = "0"
```

`amountInput` хранит текущее значение суммы счёта. Это **State** приложения для суммы счета.

10.2. Связываем State с TextField

В `TextField` задайте параметру `value` значение переменной `amountInput`:

```
TextField(
    value = amountInput,
    onValueChange = {},
```

Теперь текстовое поле отображает то значение, которое хранится в `amountInput`.

10.3. Проверка работы

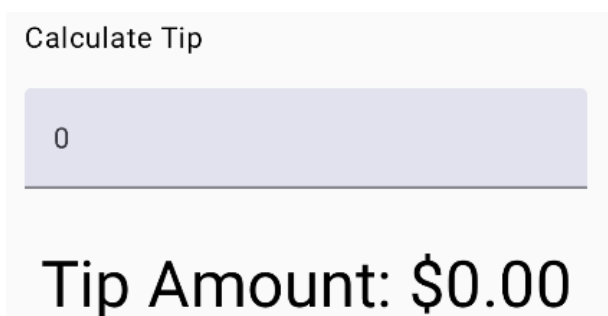
1. Проверьте **Preview**:

- В текстовом поле должно отображаться значение `"0"` — это текущее значение состояния.

2. Запустите приложение в **эмуляторе** и попробуйте ввести другое число.

Результат:

- Введённое значение **не сохраняется**.
- Поле продолжает отображать `"0"`.



10.4. Почему это происходит

`amountInput` — обычная переменная, она не обновляется при вводе. Пока `onValueChange` не изменяет значение, State остаётся неизменным.

Текущее состояние:

- значение нельзя изменить;
- ввод пользователя не влияет на интерфейс.

Для корректной работы State должен быть наблюдаемым (MutableState).

Шаг 11. Композиция

Интерфейс состоит из заголовка, текстового поля и текста суммы чаевых.

11.1. Композиция

Compose выполняет компокуемые функции и формирует интерфейс на основе текущих данных. Интерфейс описывается декларативно — через состояние, а не пошаговые изменения.

11.2. Рекомпозиция

При изменении **State** Compose автоматически пересоздаёт нужные части интерфейса.

11.3. Отслеживание изменений

Compose запоминает используемые значения **State** и запускает рекомпозицию при их изменении. Единственный способ обновить интерфейс — изменить **State**.

11.4. Использование MutableState

Обычные переменные не отслеживаются Compose. `mutableStateOf()` создаёт наблюдаемое и изменяемое состояние.

1. В `EditNumberField()` измените объявление переменной:

```
var amountInput: MutableState<String> = mutableStateOf("0")
```

Или сокращённый вариант с выводом типа:

```
var amountInput = mutableStateOf(value = "0")
```

2. Подключите необходимые импорты:

```
import androidx.compose.runtime.MutableState
import androidx.compose.runtime.mutableStateOf
```

Функция `mutableStateOf("0")`:

- создаёт наблюдаемое состояние,
- делает его **изменяемым**,
- позволяет Compose **отслеживать изменения значения**.

После этого Android Studio покажет предупреждение: *Creating a state object during composition without using remember.* Это нормально на текущем этапе — исправим в следующем шаге.

```
MainActivity.kt C:\Users\Paltos\AndroidStudioProjects\Tip_Time_Theme\app\src
! Creating a state object during composition without using `remember` :86
! None of the following candidates is applicable:... :87
```

11.5. Используем value в TextField

В TextField замените **amountInput** на **amountInput.value**:

```
TextField(
    value = amountInput.value,
```

11.6. Обновление State при вводе

В параметре **onValueChange** обновите значение состояния:

```
onValueChange = { amountInput.value = it },
```

Теперь:

- при каждом вводе текста вызывается **onValueChange**,
- новое значение сохраняется в **amountInput.value**,
- **Compose** получает сигнал об изменении **State**,
- запускается **рекомпозиция**.

Проблема:

- State обновляется;
- рекомпозиция работает;
- но значение сбрасывается, так как переменная создаётся заново при каждом вызове функции.

Шаг 12. Используйте функцию запоминания для сохранения состояния

Проблема: при рекомпозиции компонуемые функции вызываются заново, и без сохранения **State** значение сбрасывается. Для сохранения состояния между рекомпозициями используется **remember**.

12.1. Для чего нужна remember

remember сохраняет значение при первой композиции и возвращает его при последующих **рекомпозициях**, предотвращая сброс **State**. Чаще всего используется вместе с **mutableStateOf**.

12.2. Обновляем объявление переменной amountInput

1. В функции **EditNumberField()** замените старое объявление amountInput на новое, с использованием **remember** и делегирования **by**:

```
var amountInput by remember { mutableStateOf( value = "0") }
```

Теперь:

- "" (пустая строка) — это **начальное значение состояния**;
- **remember** сохраняет это значение между рекомпозициями;
- **by** позволяет работать с переменной **без .value**.

12.3. Подключаем необходимые импорты

Добавьте в начало файла:

```
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.runtime.getValue
```

Эти импорты позволяют:

- читать значение **amountInput** напрямую,
- изменять его без обращения к **value**.

12.4. Итоговый вид функции EditNumberField()

После всех изменений функция должна выглядеть так:

```
@Composable
fun EditNumberField(modifier: Modifier = Modifier) {
    var amountInput by remember { mutableStateOf( value = "0") }
    TextField(
        value = amountInput,
        onValueChange = { amountInput = it },
        modifier = modifier
    )
}
```

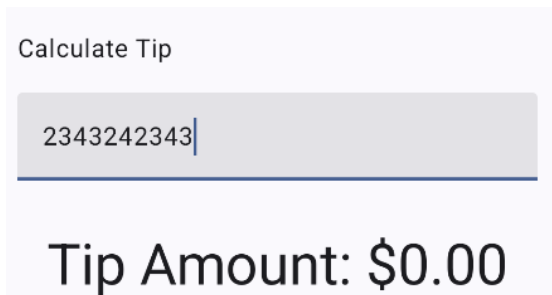
12.5. Как теперь работает состояние

- значение инициализируется один раз;
- при вводе обновляется через onValueChange;
- Compose запускает рекомпозицию;
- значение не сбрасывается, так как сохранено через remember.

12.6. Проверка работы

1. Запустите приложение.
2. Введите текст в текстовое поле.

Теперь введённое значение **не исчезает**, а корректно сохраняется и отображается.



Шаг 13. State и рекомпозиция в действии (отладка)

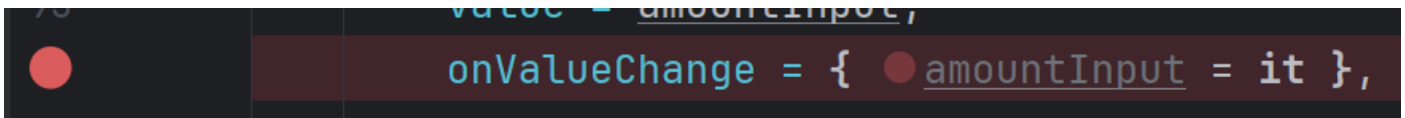
На этом шаге мы **на практике увидим**, как работают:

- начальная композиция,
- рекомпозиция,
- обновление State.

Для этого будем использовать **отладчик (Debug)**.

13.1. Установка точки останова

1. В функции `EditNumberField()` установите **точку останова** на строке `onValueChange`:



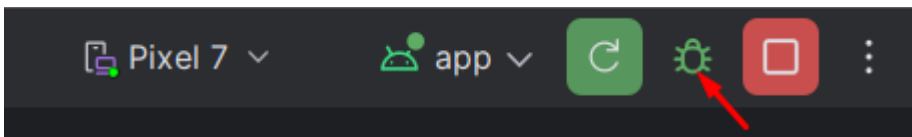
13.2. Запуск приложения в режиме отладки

2. В верхнем меню Android Studio нажмите:

Run → Debug 'app'

Приложение запустится на эмуляторе или реальном устройстве.

Выполнение программы **остановится в момент создания TextField**.



13.3. Продолжение выполнения

3. В панели **Debug** нажмите:

Resume Program 

Текстовое поле будет создано и отображено в приложении.

13.4. Проверка рекомпозиции при вводе

4. Введите любой символ в текстовое поле.

В этот момент:

- срабатывает `onValueChange`,
- выполнение программы **снова останавливается** на точке останова,
- в переменной `it` вы увидите **введённый символ**.

Это и есть новое значение `State`.

13.5. Продолжаем выполнение

5. Нажмите **Resume Program**  ещё раз.

Введённый текст:

- сохранится в `amountInput`,
- после рекомпозиции **отобразится в `TextField`**.

6. Нажмите **Resume Program**  ещё раз — приложение продолжит обычную работу.

Шаг 14. Изменение внешнего вида

На прошлых шагах вы настроили работу текстового поля. Теперь улучшим **внешний вид и удобство ввода данных**.

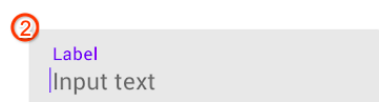
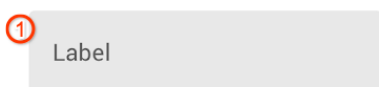
14.1. Добавление метки (label)

Каждое текстовое поле должно иметь **метку**, которая подсказывает пользователю, какие данные нужно вводить.

Что делает метка:

- пока поле не активно — текст метки находится внутри поля;
- при нажатии — метка переносится вверх.

В первой части следующего примера изображения текст метки находится посередине текстового поля и выровнен по строке ввода. Во второй части следующего примера изображения метка перемещается выше в текстовом поле, когда пользователь щёлкает по нему для ввода текста.



1. Добавляем параметр label в TextField

В функции EditNumberField() в TextField() добавьте параметр label:

```
TextField(  
    value = amountInput,  
    onChange = { amountInput = it },  
    modifier = modifier,  
    label = { }  
)
```

2. Внутри label вызываем Text() со строковым ресурсом

Замените пустое тело label на:

```
label = { Text(text = stringResource(id = R.string.bill_amount)) }
```

Теперь в текстовом поле появится метка **Bill Amount**.

14.2. Ограничение ввода одной строкой

3. В TextField() добавьте параметр:

```
label = { Text(text = stringResource(id = R.string.bill_amount)) },  
singleLine = true,
```

Это:

- запрещает многострочный ввод;
- делает поле компактным;
- позволяет прокручивать ввод по горизонтали.

14.3. Настройка клавиатуры для ввода чисел

Android позволяет указывать, **какая клавиатура** будет появляться при вводе.

4. Добавьте параметр **keyboardOptions**:

```
singleLine = true,  
keyboardOptions = KeyboardOptions(),
```

Необходимый импорт:

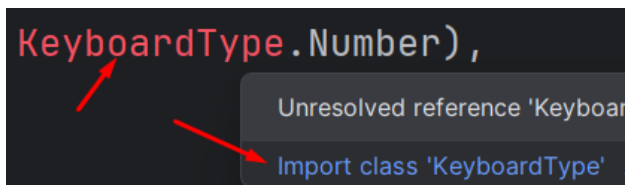
```
KeyboardOptions(),  
Unresolved reference 'KeyboardO  
Import class 'KeyboardOptions'
```

5. Устанавливаем цифровую клавиатуру

Замените KeyboardOptions() на:

```
keyboardOptions = KeyboardOptions(keyboardType = TextInputType.Number),
```

Добавьте необходимый импорт:

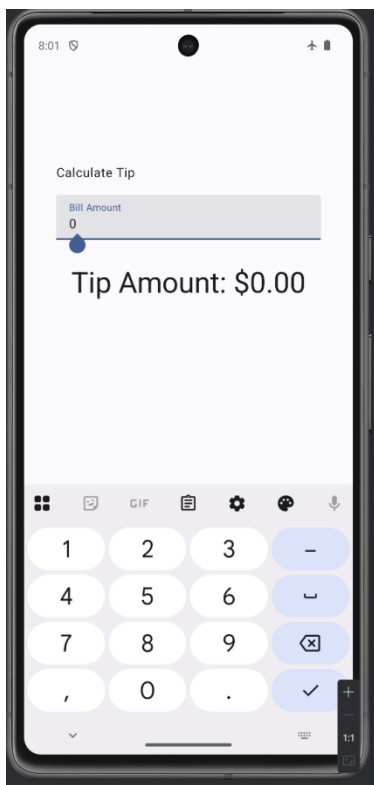


14.4. Проверка

6. Запустите приложение.

Вы должны увидеть:

- метку **Bill Amount** внутри поля;
- числовую клавиатуру при вводе;
- одну строку ввода.



Шаг 15. Отображение суммы чаевых

На этом шаге мы реализуем **основную функциональность приложения** — расчёт и отображение суммы чаевых.

У нас уже есть готовая функция:

```
private fun calculateTip()
```

Теперь нужно:

1. Преобразовать пользовательский ввод в число;

2. Передать его в `calculateTip()`;
3. Отобразить результат в интерфейсе.

15.1. Преобразование ввода пользователя в число

Текст из `TextField` всегда приходит как **String**, даже если пользователь вводит цифры. Поэтому его нужно преобразовать в `Double`.

1. Создание переменной `amount`

В функции `EditNumberField()` после объявления `amountInput` создайте переменную `amount`:

```
fun EditNumberField(modifier: Modifier = Modif
    var amountInput by remember { mutableState
    val amount = amountInput.toDoubleOrNull()
```

Функция `toDoubleOrNull()`:

- пытается преобразовать строку в число;
- возвращает `null`, если введен некорректный символ.

2. Защита от `null` (оператор Элвиса)

Добавьте оператор `?:`, чтобы избежать ошибки:

```
val amount = amountInput.toDoubleOrNull() ?: 0.0
```

Примечание : `?:` Оператор Elvis возвращает выражение, предшествующее ему, если значение **не равно** выражению `null`, и предшествующее ему, если значение **равно** `null`. Это позволяет записать этот код более идиоматично.

3. Вычисляем сумму чаевых

Создайте переменную `tip`, используя вашу функцию:

```
val amount = amountInput.toDouble
val tip = calculateTip(amount)
```

Теперь `tip` содержит отформатированную строку суммы чаевых.

15.2. Отображение суммы чаевых в интерфейсе

В `TipTimeLayout()` уже есть текст:

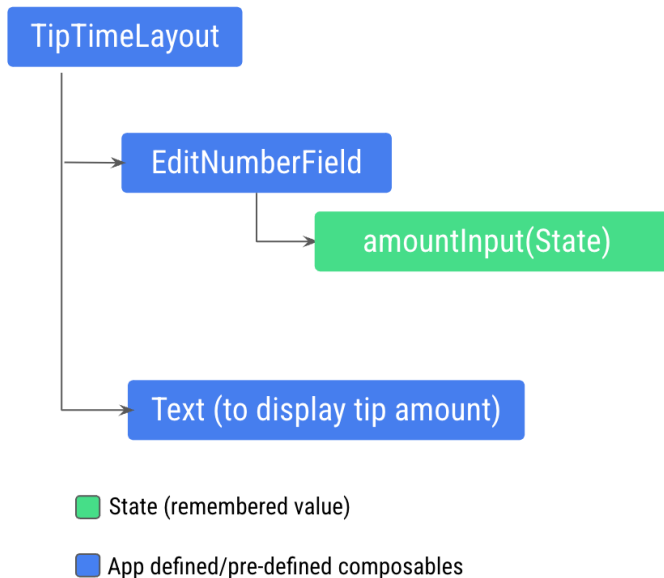
```
Text(
    text = stringResource(id = R.string.tip_amount, ...formatArgs = "$0.00"),
    style = MaterialTheme.typography.displaySmall
```

Именно это место будет отображать вычисленные чаевые.

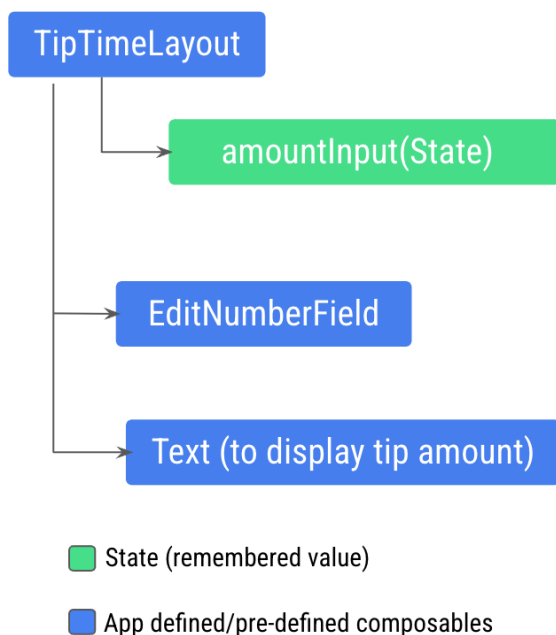
Однако сейчас возникает **архитектурная проблема**:

- `amountInput` и `amount` находятся в `EditNumberField()`,
- а `Text` с `Tip Amount` находится в `TipTimeLayout()`,
- одна функция **не видит состояние другой**.

На этом изображении показана структура кода:



Эта структура не позволит вам отобразить сумму чаевых в новом `Text` объекте, поскольку ему необходим доступ к переменной **`amount`**, вычисленной на основе этой переменной `amountInput`. Вам необходимо предоставить переменную `amount` функции `TipTimeLayout()`. На этом изображении показана желаемая структура кода, которая делает компонент `EditNumberField()` не сохраняющим состояние:



Т.е. нужно **поднять состояние вверх**.

- **amountInput** должен храниться в **TipTimeLayout()**,
- **EditNumberField()** должен просто **принимать значение и обновлять его**.

Такой подход называется:

Подъём состояния (State Hoisting)

Важно:

Компонуемый объект без состояния (stateless) — это объект, который:

- не хранит своё собственное состояние;
- получает его через параметры;
- сообщает об изменениях через **onValueChange**.

Именно к этому мы перейдём **на следующем шаге**.

Шаг 16. State hoisting

Ранее состояние **amountInput** хранилось внутри **EditNumberField()**. Но теперь к нему должен иметь доступ **TipTimeLayout()**, чтобы отображать сумму чаевых. Поэтому состояние нужно **поднять**.

Когда нужно поднимать состояние

Подъём состояния используют, когда нужно:

- разделить состояние между несколькими компонуемыми функциями;
- сделать компонент без собственного состояния.

Компонент без состояния (stateless):

- не хранит состояние;
- получает данные через параметры;
- сообщает об изменениях через **onValueChange**.

16.1. Обновляем EditNumberField() — делаем его без состояния

1. Измените сигнатуру функции:

```
@Composable
fun EditNumberField(
    value: String,
    onValueChange: (String) -> Unit,
    modifier: Modifier = Modifier
) {...}
```

Теперь:

- **value** — текущее значение поля;
- **onValueChange** — колбэк для обновления значения.

2. Внутри **TextField()** используйте переданные параметры:

```
TextField(
    value = value,
    onValueChange = onValueChange,
    modifier = modifier,
```

Теперь **EditNumberField()** не хранит состояние.

16.2. Поднимаем состояние в **TipTimeLayout()**

3. Перенесите **remember** из **EditNumberField()** в **TipTimeLayout()** и установите для значения **amountInput** пустую строку:

```
@Composable
fun EditNumberField(
    value: String,
    onValueChange: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    // var amountInput by remember { mutableStateOf("0") }
    // val amount = amountInput.toDoubleOrNull() ?: 0.0
    // val tip = calculateTip(amount)
    TextField(
```

```
@Composable
fun TipTimeLayout() {
    var amountInput by remember { mutableStateOf(value = "") }
    val amount = amountInput.toDoubleOrNull() ?: 0.0
    val tip = calculateTip(amount)
    Column(
```

Теперь владельцем состояния является **TipTimeLayout()**.

16.3. Передаём состояние обратно в **EditNumberField()**

4. Обновите вызов **EditNumberField()** внутри **TipTimeLayout()**:

```
Column(...) {
    Text(...)
    EditNumberField(
        value = amountInput,
        onValueChange = { amountInput = it },
        modifier = Modifier
            .padding(bottom = 32.dp)
            .fillMaxWidth()
    )
    Text(...)
    Spacer(modifier = Modifier.height(height = 150.dp))
```

Теперь:

- `TipTimeLayout()` хранит состояние;
- `EditNumberField()` только отображает и передаёт изменения.

16.4. Отображение чаевых через строковый ресурс

В `strings.xml` уже используется позиционное форматирование:

```
<string name="tip_amount">Tip Amount: %s</string>
```

Раньше было:

```
Text(  
    text = stringResource(id = R.string.tip_amount, ...formatArgs = "$0.00"),  
    style = MaterialTheme.typography.displaySmall  
)
```

Теперь используем вычисленное значение `tip`:

```
Text(  
    text = stringResource(id = R.string.tip_amount, ...formatArgs = tip),  
    style = MaterialTheme.typography.displaySmall  
)
```

Итоги:

- подняли состояние `amountInput` в `TipTimeLayout()`;
- сделали `EditNumberField()` компонентом **без состояния**;
- связали ввод пользователя с расчётом чаевых;
- начали корректно отображать результат через `stringResource`.

Финальная проверка

1. Запустите приложение.
2. Введите сумму счёта.
3. Сумма чаевых (15%) должна обновляться **в реальном времени**.

Calculate Tip

Bill Amount
50

Tip Amount: \$7.50

Шаг 17. Добавление поля для процента чаевых

В этом шаге вы добавите **второе текстовое поле** — для ввода процента чаевых.

17.1. Добавляем строковые ресурсы

Откройте `res` → `values` → `strings.xml` и добавьте:

```
<string name="how_was_the_service">Tip Percentage</string>
<string name="round_up_tip">Round up tip?</string>
```

17.2. Делаем `EditNumberField()` полностью переиспользуемым

Ранее метка была жёстко прописана. Теперь передаём её как параметр.

1. Добавьте параметр **label** в сигнатуру функции:

```
@Composable
fun EditNumberField(
    + label: Int,
    value: String,
```

2. Замените жёсткую строку в `TextField`:

Было:

```
label = { Text(text = stringResource(id = R.string.bill_amount)) },
```

Стало:

```
label = { Text(text = stringResource(id = label)) },
```

3. Чтобы обозначить, что параметр `label` должен быть ссылкой на строковый ресурс, аннотируйте параметр функции с помощью `@StringRes` аннотации:

```
@Composable
fun EditNumberField(
    → @StringRes label: Int,
```

4. Импортируйте

```
import androidx.annotation.StringRes
```

Примечание: Эта `@StringRes` аннотация — типобезопасный способ использования строковых ресурсов. Она указывает, что передаваемое целое число является строковым ресурсом из `values/strings.xml` файла.

17.3. Обновляем существующее поле суммы

В `TipTimeLayout()` обновите вызов первого поля:

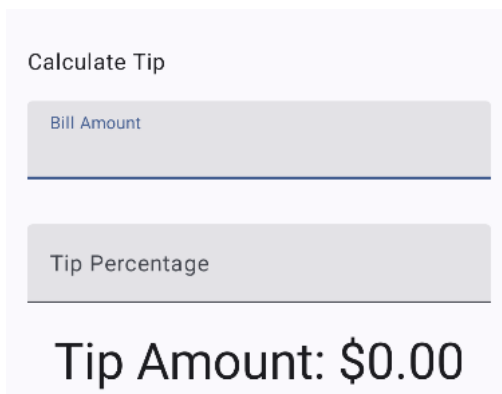
```
EditNumberField(  
    + label = R.string.bill_amount,  
    value = amountInput,
```

17.4. Добавляем второе поле — процент чаевых

Добавьте под первым полем:

```
EditNumberField(...)  
EditNumberField(  
    label = R.string.how_was_the_service,  
    value = "",  
    onValueChange = { },  
    modifier = Modifier  
        .padding(bottom = 20.dp)  
        .fillMaxWidth()  
)  
Text(...)  
Spacer(modifier = Modifier.height(height = 150.dp))
```

Теперь появится второе текстовое поле **Tip Percentage**.



The screenshot shows a mobile app interface with a title "Calculate Tip". Below the title are two input fields: "Bill Amount" and "Tip Percentage". Below these fields, the text "Tip Amount: \$0.00" is displayed.

17.5. Добавляем состояние для процента чаевых

В начале `TipTimeLayout()`:

```
fun TipTimeLayout() {  
    var amountInput by remember { mutableStateOf(value = "") }  
    val amount = amountInput.toDoubleOrNull() ?: 0.0  
    val tip = calculateTip(amount)  
    + var tipInput by remember { mutableStateOf(value = "") }  
}
```

17.6. Привязываем состояние ко второму полю

Обновите вызов второго `EditNumberField()`:

```

EditNumberField(
    label = R.string.how_was_the_service,
    value = tipInput,
    onValueChange = { tipInput = it },

```

17.7. Преобразуем процент в число

После **tipInput** добавьте:

```

@Composable
fun TipTimeLayout() {
    var amountInput by remember { mutableStateOf(value = "") }
    val amount = amountInput.toDoubleOrNull() ?: 0.0
    val tip = calculateTip(amount)
    var tipInput by remember { mutableStateOf(value = "") }
    → val tipPercent = tipInput.toDoubleOrNull() ?: 0.0

```

17.8. Обновляем расчёт чаевых

Передайте процент во второй параметр calculateTip:

```

fun TipTimeLayout() {
    var amountInput by remember { mutableStateOf(value = "0") }
    val amount = amountInput.toDoubleOrNull() ?: 0.0
    var tipInput by remember { mutableStateOf(value = "") }
    val tipPercent = tipInput.toDoubleOrNull() ?: 0.0
    → val tip = calculateTip(amount, tipPercent)

```

Финальная проверка

1. Запустите приложение.
2. Введите сумму счёта.
3. Введите процент чаевых.
4. Проверьте — **сумма чаевых должна корректно пересчитываться**

Calculate Tip

Bill Amount

50

Tip Percentage

5

Tip Amount: \$2.50

Шаг 18. Добавляем кнопку действия

Каждое текстовое поле может задавать свою кнопку действия на клавиатуре:

- **Next** — перейти к следующему полю
- **Done** — завершить ввод и закрыть клавиатуру

В этом шаге вы настроите:

- для *Bill Amount* → **Next**
- для *Tip Percentage* → **Done**

18.1. Задаём кнопку «Next» для первого поля

Сначала добавим **IME Action** напрямую внутрь **EditNumberField**, чтобы увидеть результат.

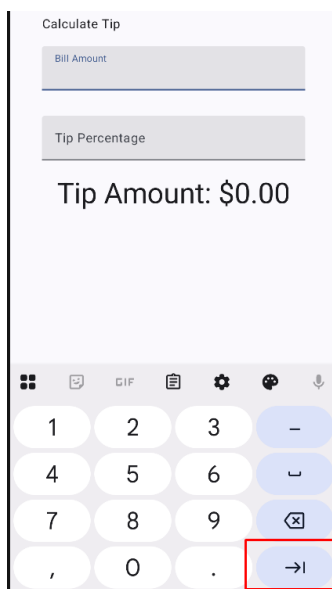
Внутри функции **EditNumberField()**:

```
@Composable
fun EditNumberField(
    ...
) {
    TextField(
        ...
        keyboardOptions = KeyboardOptions.Default.copy(
            keyboardType = KeyboardType.Number,
            imeAction = ImeAction.Next),
    )
}
```

Импорт:

```
import androidx.compose.ui.text.input.ImeAction
```

Запустите приложение — обе клавиатуры покажут *Next*, что нам пока не подходит.



18.2. Делаем IME-настройку параметром функции

Чтобы у каждого поля была своя кнопка, вынесем `keyboardOptions` в аргументы.

1. Добавьте параметр в `EditNumberField()`:

```
@Composable
fun EditNumberField(
    @StringRes label: Int,
    + keyboardOptions: KeyboardOptions,
    value: String,
```

И в теле функции присвойте его именованному параметру функции `TextField()` параметр `keyboardOptions`:

```
@Composable
fun EditNumberField(
    ...
) {
    TextField(
        ...
        keyboardOptions = keyboardOptions
    )
}
```

18.3. Настраиваем кнопки действий в функции `TipTimeLayout()`

1. В первом вызове функции `EditNumberField()` обновим поле «Bill Amount» на Next:

```
1 EditNumberField(
    label = R.string.bill_amount,
    keyboardOptions = KeyboardOptions.Default.copy(
        keyboardType = KeyboardType.Number,
        imeAction = ImeAction.Next
    ),
    value = amountInput,
```

2. Во втором вызове функции `EditNumberField()` обновим поле «Tip Percentage» на Done:

```

2 EditNumberField(
    label = R.string.how_was_the_service,
    keyboardOptions = KeyboardOptions.Default.copyWith(
        keyboardType = KeyboardType.Number,
        imeAction = ImeAction.Done
    ),
    value = tipInput,

```

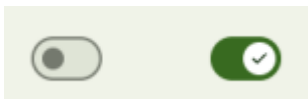
18.4. Проверка

1. Запустите приложение на эмуляторе.
2. В первом поле появится кнопка **Next**.
3. Во втором — кнопка **Done**.
4. Нажмите **Next** → курсор перейдёт к следующему полю.
5. Нажмите **Done** → клавиатура закроется.

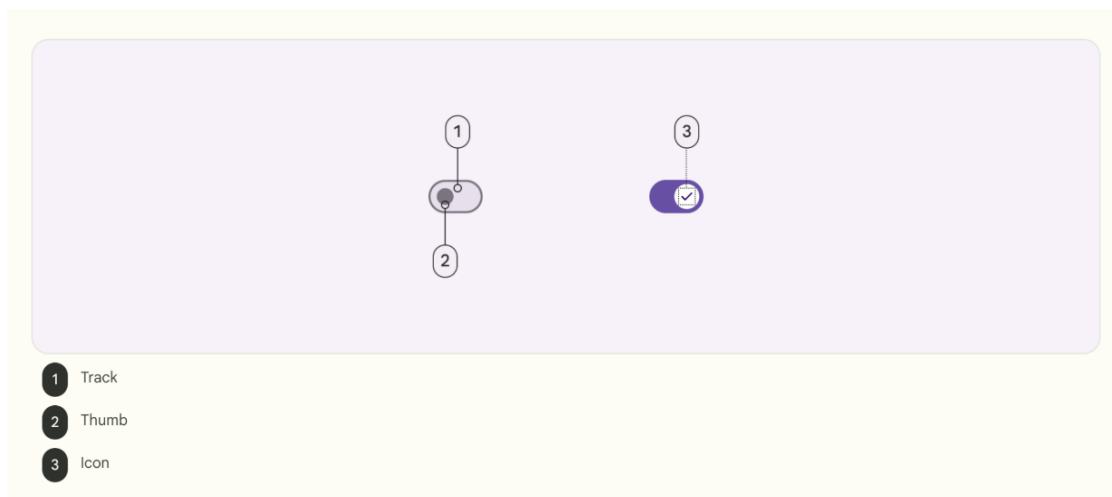


Шаг 19. Добавляем переключатель

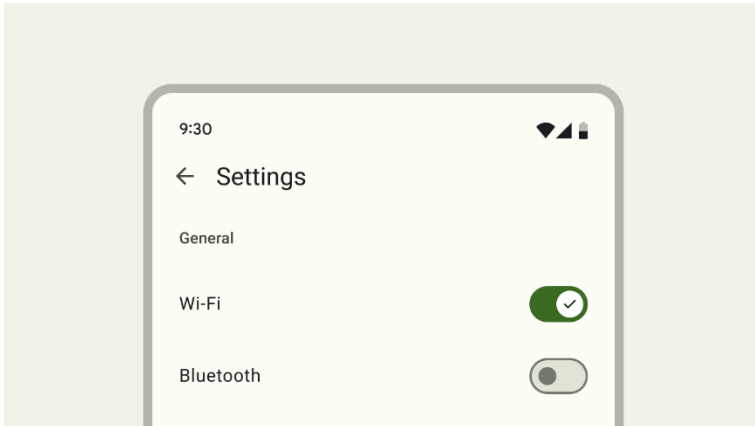
Переключатель нужен для того, чтобы включить или выключить состояние одного элемента.



Переключатель имеет два состояния, позволяющих пользователю выбрать между двумя вариантами. Переключатель состоит из трека, ползунка и опционального значка, как показано на этих изображениях:



Переключатель — это элемент управления, который можно использовать для ввода решений или объявления предпочтений, таких как настройки, как вы можете видеть на этом изображении:



Создание компонуемой функции для строки с переключателем

Создайте новую `@Composable`-функцию **RoundTheTipRow** (`modifier: Modifier = Modifier`) — она будет отображать строку текста и `Switch` в один ряд.

```
@Composable
fun RoundTheTipRow(modifier: Modifier = Modifier) {

}
```

Внутри:

- используйте `Row`, чтобы элементы шли по горизонтали;
- выровняйте содержимое по центру по вертикали;
- задайте фиксированную высоту (`48.dp`) для строки;

```
@Composable
fun RoundTheTipRow(modifier: Modifier = Modifier) {
    Row(
        modifier = modifier
            .fillMaxWidth()
            .size(size = 48.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
    }
}
```

Импорты:

```
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.size
```

В лямбда-блоке **Row()** компонуемого макета добавьте **компонуемый объект Text**, который использует строковый ресурс для отображения строки:

```
fun RoundTheTipRow(modifier: Modifier = Modifier) {  
    Row(  
        ...  
    ) {  
        Text(text = stringResource(id = R.string.round_up_tip))  
    }  
}
```

Параметры функции RoundTheTipRow()

В функции **RoundTheTipRow()** добавьте параметр **roundUp** Boolean типа и лямбда-функцию **onRoundUpChanged**, которая принимает Boolean и ничего не возвращает:

```
@Composable  
fun RoundTheTipRow(  
    roundUp: Boolean,  
    onRoundUpChanged: (Boolean) -> Unit,  
    modifier: Modifier = Modifier  
) {  
    Row(  
        ...  
    ) {  
        Text(text = stringResource(id = R.string.round_up_tip))  
        Switch(  
            checked = roundUp,  
            onCheckedChange = onRoundUpChanged,  
        )  
    }  
}
```

После **компонуемого объекта Text** добавьте компонуемый объект **Switch** и передайте именованные параметры:

```
@Composable  
fun RoundTheTipRow(  
    ...  
) {  
    Row(  
        ...  
    ) {  
        Text(text = stringResource(id = R.string.round_up_tip))  
        Switch(  
            checked = roundUp,  
            onCheckedChange = onRoundUpChanged,  
        )  
    }  
}
```

Импорт:

```
import androidx.compose.material3.Switch
```

- **checked** - проверяет отмечен ли переключатель. Это состояние компонентного объекта Switch.
- **onCheckedChange** - обратный вызов, который будет вызван при нажатии переключателя.

Чтобы выровнять **Switch** по краю экрана, добавьте modifier:

```
Switch(  
    modifier = modifier  
        .fillMaxWidth()  
        .wrapContentWidth(Alignment.End),  
    checked = roundUp,  
    onCheckedChange = onRoundUpChanged,  
)
```

Импортируйте:

```
.wrapContentWidth(Alignment.End),  
checked = r  
checkedCh
```

Unresolved reference 'wrapContentWidth'.
Import extension function 'Modifier.wrapContentWidth'

В функции **TipTimeLayout()** добавьте переменную **var** для Switch состояния компонентного объекта со значением **false** по умолчанию:

```
@Composable  
fun TipTimeLayout() {  
    var roundUp by remember { mutableStateOf(value = false) }  
    var amountInput by remember { mutableStateOf(value = "") }  
}
```

Затем в нужном месте **Column** (между полями ввода и итоговым текстом) вызовите **RoundTheTipRow** (скриншот на следующей странице), передав:

- **roundUp = roundUp**
- **onRoundUpChanged = { roundUp = it }**
- и нужный **modifier** (отступ снизу).

После этого переключатель появится и будет менять значение **roundUp** в **TipTimeLayout()**.

Calculate Tip

Bill Amount

Tip Percentage

Round up tip? ☐

Tip Amount: \$0.00

```
@Composable
fun TipTimeLayout() {
    ...
    Column(
        ...
    ) {
        Text(
            ...
        )
        EditNumberField(
            ...
        )
        EditNumberField(
            ...
        )
        RoundTheTipRow(
            roundUp = roundUp,
            onRoundUpChanged = { roundUp = it },
            modifier = Modifier.padding(bottom = 32.dp)
        )
        Text(
            ...
        )
        Spacer(modifier = Modifier.height(height = 150.dp))
    }
}
```

Обновление логики расчёта (calculateTip)

Чтобы переключатель реально влиял на результат, измените **calculateTip()**:

- Добавьте новый параметр **roundUp: Boolean** (третий параметр).

```
private fun calculateTip(
    amount: Double,
    tipPercent: Double = 15.0,
     roundUp: Boolean
): String {
```

- Внутри функции сделайте **tip** изменяемой (**var tip = ...**) — сначала вычислите чаевые по проценту.

```
    ): String {
        var tip = tipPercent / 100 * amount;
```

- И далее через оператор **if** добавим условие, проверяющее значение **roundUp**:

```
    var tip = tipPercent / 100 * amount;
    if (roundUp) {
        tip = kotlin.math.ceil(x = tip)
    }
    return NumberFormat.getCurrencyInstance
```

Если `roundUp = true`, примените округление вверх (`kotlin.math.ceil`) к переменной `tip`.

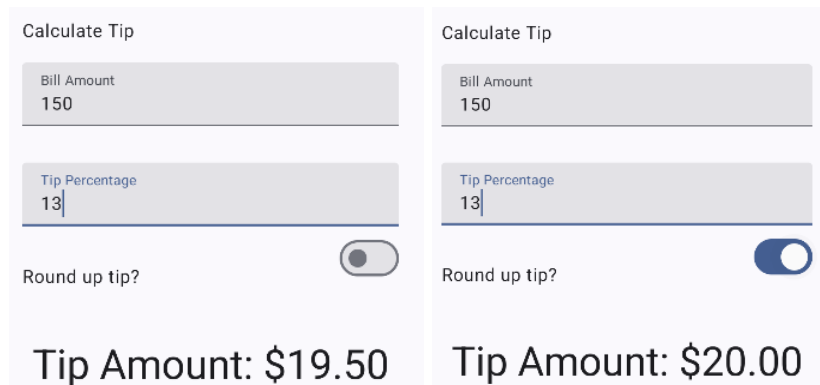
Передача состояния в вызов `calculateTip`

В функции `TipTimeLayout()` при вычислении `tip` передавайте передайте параметр **roundUp**:

```
@Composable
fun TipTimeLayout() {
    var roundUp by remember { mutableStateOf(value = false) }
    var amountInput by remember { mutableStateOf(value = "") }
    val amount = amountInput.toDoubleOrNull() ?: 0.0
    var tipInput by remember { mutableStateOf(value = "") }
    val tipPercent = tipInput.toDoubleOrNull() ?: 0.0
     val tip = calculateTip(amount, tipPercent, roundUp)
```

Проверка работы

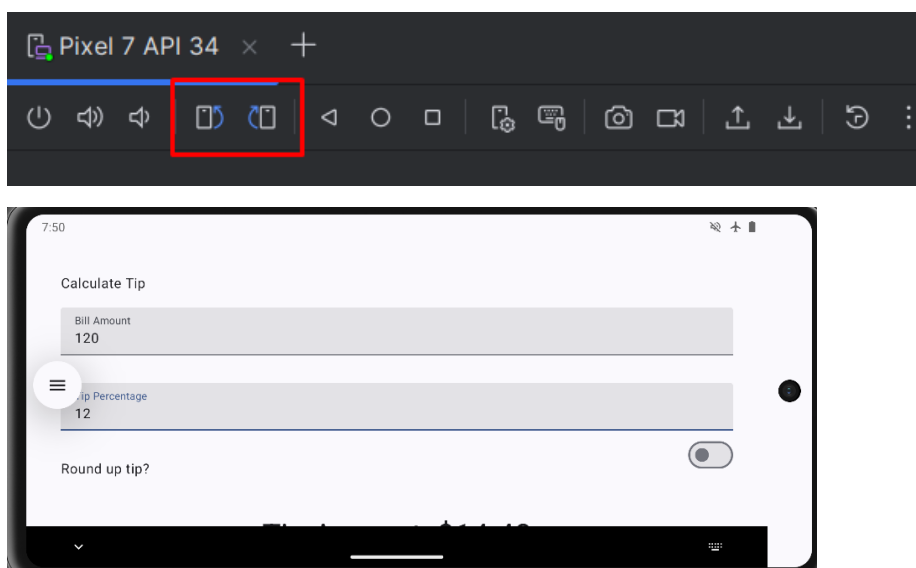
- Запустите приложение.
- Введите сумму и процент чаевых.
- Переключите «Round up tip?» в положение ON — сумма чаевых должна стать округлённой вверх.
- Переключите OFF — сумма возвращается к точному значению.



Шаг 20. Добавляем поддержку альбомной ориентации

Android-устройства имеют множество форм-факторов: телефоны, планшеты, складные и устройства на базе ChromeOS. Все они могут использовать как портретную, так и альбомную ориентацию, поэтому ваше приложение должно корректно отображаться в обеих.

Попробуйте повернуть устройство или эмулятор в альбомную ориентацию. Вы увидите, что сумма чаевых обрезается и полностью не помещается на экране.



Чтобы это исправить, нужно добавить вертикальную прокрутку. Для этого используйте модификатор `.verticalScroll(rememberScrollState())`. `rememberScrollState()` создаёт и запоминает состояние прокрутки автоматически.

После добавления вертикальной прокрутки снова запустите приложение и проверьте работу интерфейса в альбомной ориентации — теперь всё должно прокручиваться корректно.

```

@Composable
fun TipTimeLayout() {
    ...
    Column(
        modifier = Modifier
            .statusBarsPadding()
            .padding(horizontal = 40.dp)
            .safeDrawingPadding()
            .verticalScroll(rememberScrollState()),
        horizontalAlignment = Alignment.CenterHorizontally,
    ) {
        ...
    }
}

```

Добавьте необходимые **импорты**:

```

import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll

```

Запустите приложение ещё раз. Попробуйте прокрутить в альбомной ориентации!

Шаг 21. Добавление начального значка к текстовым полям

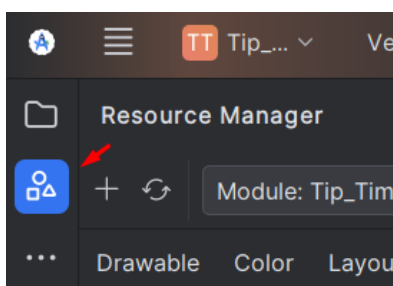
Значки делают текстовые поля более выразительными и дают пользователю подсказку о назначении поля. Например, значок денег подсказывает, что ожидается ввод суммы, а значок процента — что вводится процентное значение.

1. Импорт изображений в проект

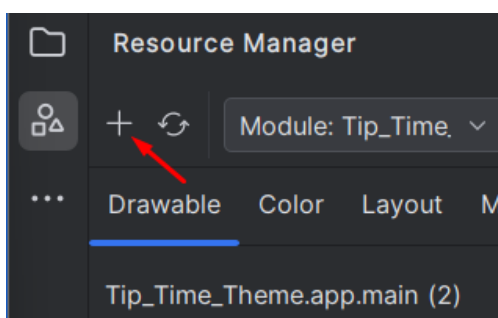
Скачайте две иконки на свой выбор: **money** и **percent**.

1. Откройте **Resource Manager: View > Tool Windows > Resource Manager**

Или нажмите соответствующую вкладку:



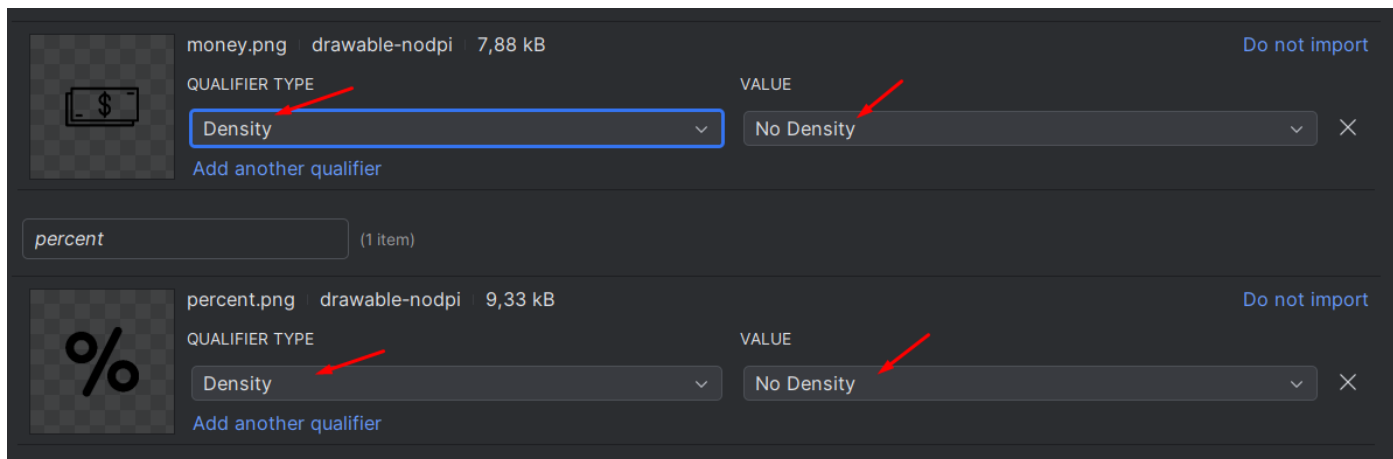
2. Нажмите + → **Import Drawables**:



3. В файловом браузере выберите загруженные вами файлы изображений и нажмите «Open». Это действие открывает диалоговое окно «Import drawables».

4. В диалоговом окне выберите:

- **Qualifier Type:** Density
- **Value:** No Density (чтобы избежать автоматического масштабирования).



5. Нажмите **Next -> Import**:

2. Добавление значков в текстовые поля

Чтобы использовать значок как начальный (leading icon), выполните следующее:

1. Добавьте в функцию **EditNumberField()** новый параметр **leadingIcon** типа **Int**, аннотированный **@DrawableRes**.

```
@Composable
fun EditNumberField(
    @StringRes label: Int,
    + @DrawableRes leadingIcon: Int,
    keyboardOptions: KeyboardOptions,
```

Добавьте импорты:

```
import androidx.annotation.DrawableRes
import androidx.compose.material3.Icon
```

2. Добавьте начальный значок в текстовое поле **TextField()** у компонуемого объекта **EditNumberField()** и сразу зададим ему размер, чтобы иконки не были огромными:

```

@Composable
fun EditNumberField(
    ...
) {
    TextField(
        value = value,
        leadingIcon = {
            Icon(
                painter = painterResource(id = leadingIcon),
                contentDescription = null,
                modifier = Modifier.size(size = 24.dp)
            )
        },
        onChange = onChange,
    )
}

```

3. Передайте значок для первого поля (сумма счёта) в функции **TipTimeLayout()**:

```

EditNumberField(
    label = R.string.bill_amount,
    + leadingIcon = R.drawable.money,
    keyboardOptions = KeyboardOptions.Default.copy(

```

4. Передайте значок для второго поля (процент чаевых) в функции **TipTimeLayout()**:

```

EditNumberField(
    label = R.string.how_was_the_service,
    + leadingIcon = R.drawable.percent,
    keyboardOptions = KeyboardOptions.Default.copy(

```

Перезапустите приложение и проверьте что иконки появились:

Calculate Tip

\$

Bill Amount

%

Tip Percentage

Round up tip?

Tip Amount: \$0.00

Шаг 22. Автоматизированные тесты в Android

Теория. Что такое автоматизированные тесты

Тестирование — это способ проверить, что приложение работает так, как задумано.

Автоматизированные тесты — это код, который проверяет другой код без участия человека.

Они позволяют:

- проверять корректность логики приложения;
- находить ошибки на ранних этапах;
- быстро проверять приложение после изменений;
- снизить объём ручного тестирования.

Автоматические тесты особенно важны, когда:

- приложение становится больше;
- код часто изменяется;
- приложение используется на разных устройствах и версиях Android.

Зачем нужны автоматизированные тесты

Без тестов каждое изменение приходится проверять вручную, что со временем становится неудобно и медленно.

Автоматизированные тесты:

- выполняются быстрее, чем ручные;
- не требуют постоянного участия разработчика;
- позволяют уверенно вносить изменения в код;
- повышают стабильность приложения.

Ручное тестирование всё равно остаётся полезным, но большую часть проверок эффективнее автоматизировать.

Виды автоматизированных тестов в Android

Локальные тесты (Local tests)

- проверяют отдельные функции, классы и свойства;
- **не требуют устройства или эмулятора;**
- выполняются прямо на компьютере;
- работают быстро и потребляют мало ресурсов.

Обычно используются для проверки **бизнес-логики**, например вычислений.

Инструментальные тесты (Instrumentation tests)

- проверяют **пользовательский интерфейс**;
- запускают реальное приложение;
- имитируют действия пользователя (ввод текста, нажатия);
- выполняются на **эмуляторе или физическом устройстве**.

Такие тесты собираются в отдельный APK и запускаются вместе с основным приложением.

Шаг 23. Написание тестов

Локальный тест

Подготовка кода приложения

Чтобы функцию можно было протестировать, она должна быть доступна из тестов.

Функция **calculateTip()**:

- должна быть не **private**;
- помечается аннотацией **@VisibleForTesting**.

```
@VisibleForTesting
internal fun calculateTip(amount: Double, tipPercent: Double = 15.0,
    roundUp: Boolean): String {
```

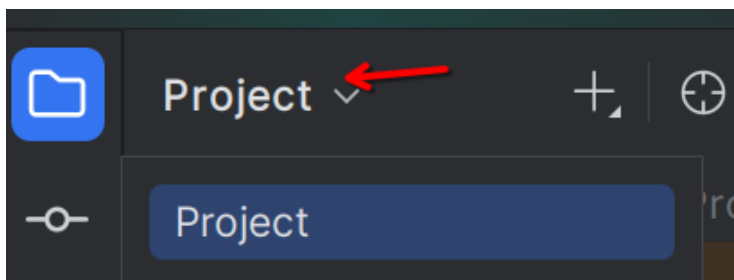
Необходимые импорты:

```
import org.jetbrains.annotations.VisibleForTesting
```

Аннотация показывает, что функция открыта **только для тестирования**.

Создание директории для локальных тестов

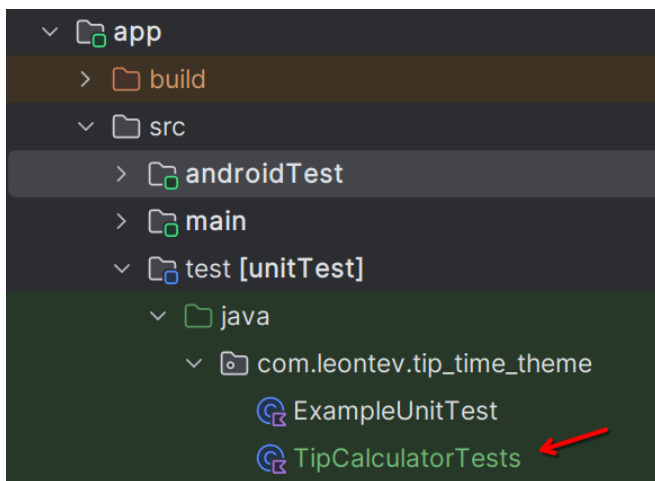
Переключиться в **Project view**



Создание тестового класса

В папке **app/src/test[unitTest]/java/com.leontev.tip_time_theme**:

- **New > Kotlin Class/File**
- имя класса: **TipCalculatorTests**



Написание локального теста

Локальный тест проверяет корректность вычисления чаевых.

Тест:

- считает 20% от 10;
- ожидаемый результат — 2.

Пишем код в нашем классе **TipCalculatorTests**:

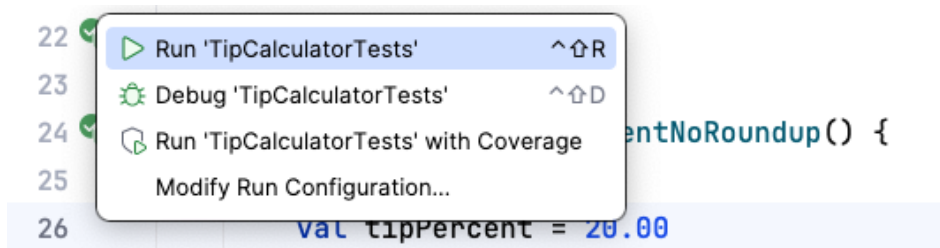
```
import org.junit.Assert.assertEquals
import org.junit.Test
import java.text.NumberFormat

class TipCalculatorTests {
    @Test
    fun calculateTip_20PercentNoRoundup() {
        val amount = 10.00
        val tipPercent = 20.00
        val expectedTip = NumberFormat.getCurrencyInstance().format(2)
        val actualTip = calculateTip(amount = amount, tipPercent =
            tipPercent, false)
        assertEquals(expectedTip, actualTip)
    }
}
```

- **@Test** — помечает метод как тест;
- **assertEquals()** — проверяет совпадение ожидаемого и фактического результата.

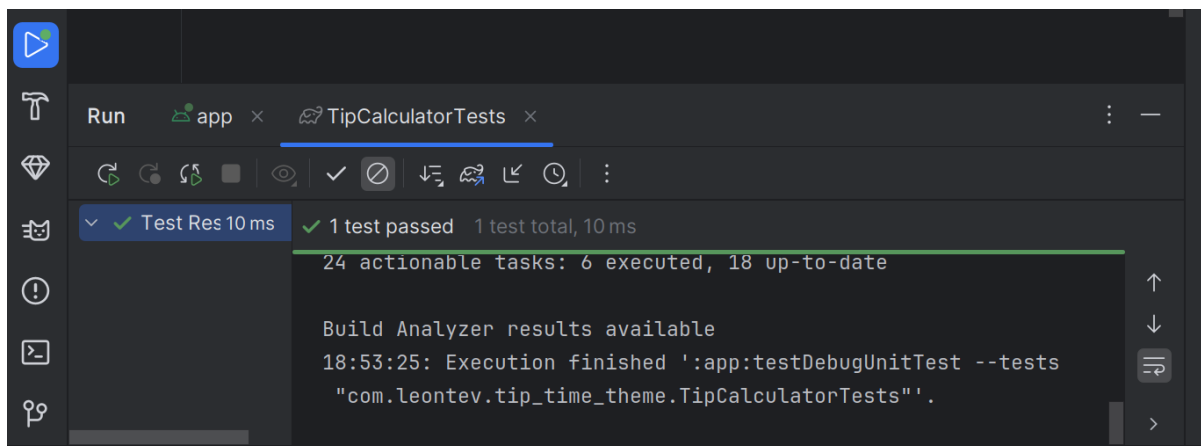
Запуск локального теста

- Нажать на зелёную стрелку рядом с классом или методом;
- Выбрать **Run 'TipCalculatorTests'**.



Вы должны увидеть следующее:

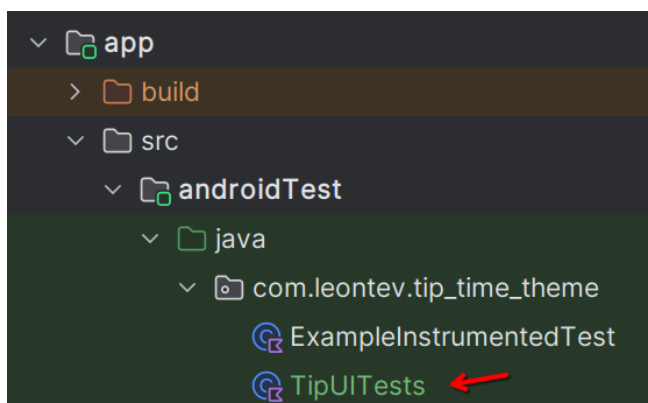
- В нижней части панели **Run** вы видите некоторую производительность.



Инструментальный (UI) тест

В папке `app/src/androidTest/java/com.leontev.tip_time_theme`:

- **New > Kotlin Class/File**
- имя класса: **TipUITests**



Написание UI-теста

Код инструментальных тестов значительно отличается от кода локальных тестов.

Инструментальные тесты проверяют **реальный экземпляр приложения и его пользовательский интерфейс**, поэтому перед выполнением тестов необходимо задать UI-контент. Это делается аналогично тому, как UI задаётся в методе `onCreate()` файла `MainActivity.kt` при разработке приложения `Tip Time`. Данный шаг обязателен для всех инструментальных тестов приложений, написанных с использованием `Compose`.

В случае приложения Tip Time тесты будут взаимодействовать с элементами интерфейса, чтобы проверить корректность вычисления чаевых **через UI**. Поначалу концепция инструментальных тестов может показаться сложной, однако далее процесс будет разобран пошагово.

1. Создание правила для Compose-тестов

Создайте переменную **composeTestRule**, присвоив ей результат вызова функции **createComposeRule()**, и пометьте её аннотацией **@Rule**:

```
import androidx.compose.ui.test.junit4.createComposeRule
import org.junit.Rule

class TipUITests {

    @get:Rule
    val composeTestRule = createComposeRule()
}
```

2. Создание тестового метода

Создайте метод **calculate_20_percent_tip()** внутри класса и пометьте его аннотацией **@Test**:

```
import org.junit.Test

class TipUITests {
    ...//код
    @Test
    fun calculate_20_percent_tip() {
    }
}
```

Компилятор автоматически определяет:

- методы с **@Test** в каталоге **androidTest** как **инструментальные тесты**;
- методы с **@Test** в каталоге **test** как **локальные тесты**.

3. Установка UI-контента в тесте

В теле тестового метода вызовите функцию **composeTestRule.setContent()**. Внутри лямбда-выражения установите тему приложения и вызовите **TipTimeLayout()**:

```
fun calculate_20_percent_tip() {
    composeTestRule.setContent {
        Tip_Time_ThemeTheme {
            TipTimeLayout()
        }
    }
}
```

Необходимые импорты:

```
import com.leontev.tip_time_theme.ui.theme.Tip_Time_ThemeTheme // ваш пакет
```

На этом этапе UI тестируемого приложения полностью инициализирован. Код аналогичен установке контента в `onCreate()` класса `MainActivity.kt`.

4. Доступ к элементам интерфейса

Элементы интерфейса в **Compose**-тестах доступны в виде **узлов (nodes)** через `composeTestRule`.

Один из наиболее распространённых способов — поиск элемента по отображаемому тексту с помощью метода `onNodeWithText()`.

Получите доступ к полю ввода суммы счёта (Bill Amount):

```
fun calculate_20_percent_tip() {
    composeTestRule.setContent {...}
    + composeTestRule.onNodeWithText("Bill Amount")
}
```

Необходимые импорты:

```
import androidx.compose.ui.test.onNodeWithText
```

5. Ввод данных в поле суммы счёта

Для ввода текста используется метод `performTextInput()`.

Заполните поле суммы значением **10**:

```
composeTestRule.onNodeWithText("Bill Amount")
    + .performTextInput("10")
```

Необходимые импорты:

```
import androidx.compose.ui.test.performTextInput
```

6. Ввод процента чаевых

Аналогичным образом заполните поле процента чаевых значением **20**:

```
composeTestRule.setContent {...}
composeTestRule.onNodeWithText("Bill Amount")
    .performTextInput("10")
+ composeTestRule.onNodeWithText("Tip Percentage")
    .performTextInput("20")
```

После заполнения всех полей приложение автоматически отображает сумму чаевых в нижней части экрана.

7. Проверка результата с помощью утверждения (assertion)

Теперь необходимо убедиться, что на экране отображается корректная сумма чаевых.

В Compose-инструментальных тестах утверждения можно вызывать непосредственно у UI-элементов. В данном случае используется метод `assertExists()`.

⚠ Символ валюты зависит от локали устройства, поэтому значение форматируется с помощью `NumberFormat`.

Утвердим, что узел с таким текстом существует:

```
composeTestRule.onNodeWithText("Bill Amount")
    .performTextInput("10")
composeTestRule.onNodeWithText("Tip Percentage")
    .performTextInput("20")
+ val expectedTip = NumberFormat.getCurrencyInstance().format(2)
  composeTestRule.onNodeWithText("Tip Amount: $expectedTip").assertExists(
    "No node with this text was found."
  )
```

Необходимые импорты:

```
import java.text.NumberFormat
```

Также добавим **Surface** для нашей функции `Tip_Time_ThemeTheme()`:

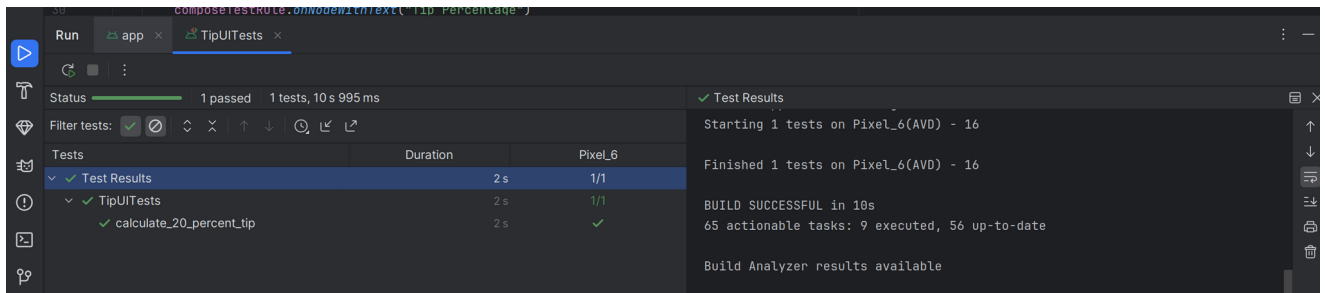
```
@Test
fun calculate_20_percent_tip() {
    composeTestRule.setContent {
        Tip_Time_ThemeTheme {
            Surface(modifier = Modifier.fillMaxSize()) {
                TipTimeLayout()
            }
        }
    }
    composeTestRule.onNodeWithText("Bill Amount")
```

Необходимые импорты:

```
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.Surface
import androidx.compose.ui.Modifier
```

Запуск UI-теста

Нажмите на стрелки рядом с объявлением класса. Вы можете увидеть тесты, проводимые на вашем устройстве или эмуляторе. Когда тест закончен, вы должны увидеть выход:



Шаг 24. Публикация проекта

Загрузите ваше приложение на **репозиторий GitHub**.

Самостоятельное задание: Оформление README.md

Создайте файл **README.md** в корне проекта, который будет описывать вашу лабораторную работу.

Требования к README.md:

Файл должен содержать следующие разделы (напишите их самостоятельно):

1. **Название проекта** — название и краткое описание приложения
2. **Цель работы** — что вы изучали в этой лабораторной
3. **Функциональность** — список реализованных возможностей приложения (что умеет делать приложение)
4. **Технологии** — какие технологии и библиотеки использовались (Jetpack Compose, Material Design 3, и т.д.)
5. **Автор** — ваше ФИО и группа

После создания **README.md**:

```
git add README.md
```

```
git commit -m "Add README documentation"
```

```
git push
```